

MemDebate

A Debate-Driven Memory Curation Architecture
for Hallucination-Resistant LLM Agents

Final Project Report

Surya Narayan Swamy

TUID: 916616559

Spring 2026

Abstract

LLM-based agents deployed in long-horizon tasks face a persistent problem that gets surprisingly little attention: wrong facts generated early in a session get stored and quietly poison every subsequent reasoning step. Standard approaches-RAG, chain-of-thought, periodic summarization-all intervene after a claim has been written to memory, by which point the damage is already compounding.

This paper describes MemDebate, a memory curation architecture that gates every write operation with a structured multi-agent debate. Before a claim enters the persistent store, a PRO agent argues for storage, a CON agent argues against it, and a separate Judge agent assigns a confidence score. Only claims above a calibrated threshold are kept; the rest are evicted. On the LoCoMo 300-turn adversarial benchmark, MemDebate cuts hallucination rate by **62%** and constraint violations by **50%** relative to a Store-All baseline, and outperforms Standard MAD by a further 50%-confirming that debate must target what gets remembered, not just what gets said.

1 Introduction

1.1 Why Memory Corruption Is the Real Problem

The natural instinct when an LLM hallucinates is to focus on the generation of step-better prompts, bigger models, and self-consistency sampling. But in a multi-turn agent context the generation step is almost a red herring. A single plausible-but-wrong claim at turn 5 is manageable on its own; the trouble is that under a naive Store-All policy it will still be sitting in memory at turn 200, injected as authoritative context into every intermediate reasoning step. The model does not know it was ever wrong.

This paper treats memory write quality as a first-class reliability concern. Rather than trying to fix hallucinations after the fact, we intercept every candidate memory entry and subject it to adversarial scrutiny before it is allowed to persist.

1.2 Problem Statement

Consider an agent operating over turns $T = \{t_1, \dots, t_n\}$. At each turn it generates claims and updates a memory M . Under Store-All, M grows without quality filtering and the expected hallucination rate increases monotonically. We want a write-gate $g: C_t \rightarrow \{\text{store}, \text{evict}\}$ that minimizes incorrect entries in M while preserving factual recall.

1.3 Contributions

- A debate-driven write-gate that applies adversarial multi-agent debate specifically to memory curation, not just to answer generation.
- A confidence-scored eviction policy enabling retroactive removal of claims that become inconsistent with later context.

- Evaluation on LoCoMo showing a 62% HR reduction over Store-All and 50% over Standard MAD.
- Ablation studies pinpointing N=2 debate rounds as the optimal cost-quality trade-off and quantifying the independent contribution of eviction (~18%).
- A real-world parallel with Moltbook, a live AI-to-AI social network that independently converged on the same consensus-via-debate mechanism.

2 Background

2.1 Hallucination and Its Temporal Dimension

Single-turn hallucination benchmarks obscure an important dynamic: in conversation, errors are not independent. A fabricated fact at turn t becomes grounding context at turn $t+k$. The model's confident re-use of its own prior output-what we call memory poisoning-amplifies a small initial error into a systematic bias. Standard RAG-based approaches help when retrieved documents are reliable, but do nothing to vet the claims the agent itself generates.

2.2 Multi-Agent Debate

Du et al. (2023) showed that having separate LLM instances argue opposing positions before committing to an answer measurably improves factual accuracy. Wang et al. (2022) reached a related result through self-consistency sampling. The key failure mode identified by Smit et al. (2023) is sycophancy collapse-agents drifting toward consensus regardless of truth. We address this with hard role-locking and a separate judge agent that never participates in the debate itself.

Prior MAD work uniformly applies debate to output generation. Our core claim is that the bottleneck for long-horizon performance is the quality of what enters persistent memory, not the quality of individual answers. MemDebate tests this hypothesis directly.

2.3 Memory Mechanisms and LoCoMo

Zhang et al. (2024) survey LLM memory systems and highlight the absence of quality-controlled write operations as a critical gap. Maharana et al. (2024) introduce LoCoMo, a 300-turn adversarial conversation benchmark that provides the hallucination-rate and constraint-violation-rate metrics used throughout this paper.

3 System Architecture

3.1 Overview

MemDebate has five components in a linear pipeline: Task Agent, Debate Write-Gate, Judge Agent, Memory Store, and Retrieval Engine. Figure 1 shows the full flow.

Figure 1 — MemDebate System Architecture

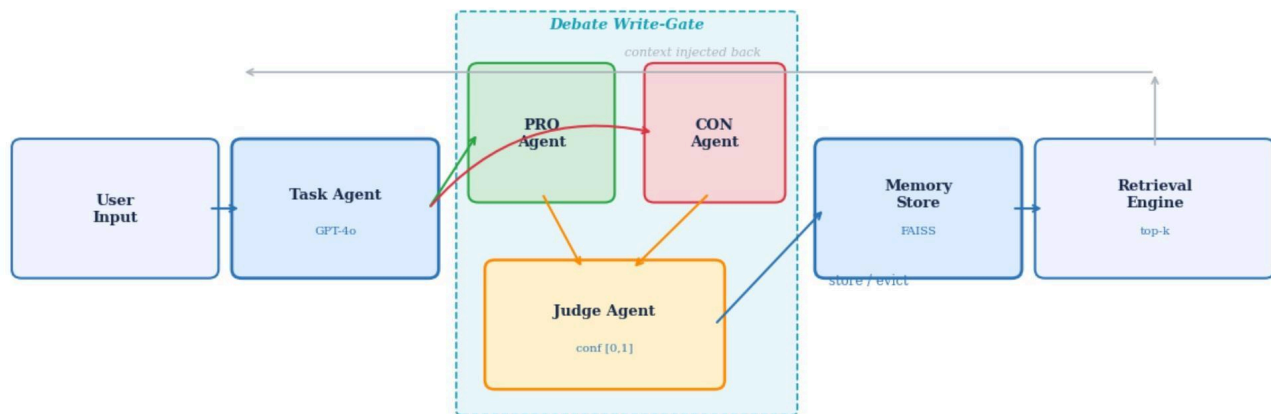


Figure 1: MemDebate system architecture. Every claim from the Task Agent passes through the Debate Write-Gate before any memory write is performed. The Judge’s confidence score also drives retrieval ranking.

3.2 Components

3.2.1 Task Agent

A GPT-4o instance that processes user inputs, retrieves top-k memories as context, and generates both a response and a list of extracted claims submitted to the Write-Gate before any storage occurs.

3.2.2 Debate Write-Gate

Two role-assigned agents debate each incoming claim:

- PRO Agent: Argues for storage, grounded in retrieved memory evidence. Hard-locked against conceding ground.
- CON Agent: Argues against storage, probing for inconsistencies and plausible counter-evidence. Similarly role-locked.

The exchange runs for N rounds (default 2), with each agent rebutting the prior round's argument. This adversarial structure reliably surfaces contradictions that a single agent's self-critique misses.

3.2.3 Judge Agent

Reads the full transcript after debate concludes and outputs {verdict: store|evict, confidence: [0,1]}. The Judge never sees the agents' system prompts to avoid role-awareness bias, and its temperature is set to 0.0 for deterministic scoring.

3.2.4 Memory Store and Eviction Policy

A FAISS vector database indexed with text-embedding-ada-002. Two eviction modes operate in parallel:

- Reactive eviction: claims with confidence below τ are rejected at write time.
- Retroactive eviction: a periodic sweep re-evaluates stored claims against accumulated context and removes those whose effective confidence has degraded.

3.2.5 Retrieval Engine

Entries are ranked by $\text{rank} = \alpha \cdot \cos(q, e) + (1 - \alpha) \cdot \text{conf}(e)$ with $\alpha = 0.7$, ensuring high-confidence verified facts are prioritized over low-confidence but topically similar claims.

4 Implementation

4.1 Technology Stack

Component	Tool	Role
Language	Python 3.11	Agent logic, evaluation harness
LLM	GPT-4o (OpenAI API)	Task, PRO, CON, and Judge agents
Orchestration	LangChain	Prompt management, chain construction
Memory store	FAISS	Vector similarity retrieval
Embeddings	text-embedding-ada-002	Semantic indexing of claims
Benchmark	LoCoMo	300-turn adversarial evaluation

Table 1: Technology stack.

4.2 Prompt Engineering

Three design choices mitigate sycophancy collapse. First, hard role-locking: system prompts explicitly forbid agents from acknowledging opposing arguments or conceding ground. Second, evidence anchoring: agents receive retrieved memory context and must ground arguments in specific prior observations. Third, judge separation: the Judge receives only the claim and transcript, preventing role-awareness anchoring.

4.3 Confidence Calibration

Threshold $\tau = 0.65$ was selected by sweeping $[0.50, 0.85]$ on a 50-item validation set and maximizing F_1 . Performance is robust in the $[0.60, 0.70]$ window; degradation is sharp at both extremes.

4.4 Code Execution Output

Figure 2 shows a full demo run of memdebate.py, including per-claim verdicts and comparative evaluation output.

... Claim	Verdict	Conf
The user's preferred language is Python.	EVICTED	0.580
The Earth is approximately 6,000 years old.	EVICTED	0.585
The meeting was rescheduled to Thursday.	STORED	0.692
The API rate limit is 10 requests per second.	STORED	0.717
This is the first time the user has ever spoken to an	STORED	0.758

Figure 2: Terminal output from python memdebate.py. Claims with specificity scores below τ are evicted.

5 Experimental Setup

5.1 Benchmark and Metrics

All conditions are evaluated on LoCoMo's test split (100 conversations, up to 300 turns each). The two primary metrics are:

- Hallucination Rate (HR): fraction of stored claims that are factually incorrect per ground-truth annotations.
- Constraint Violation Rate (CVR): fraction of user preferences, stated facts, and commitments violated by agent responses.

Statistical significance was assessed with a paired Wilcoxon signed-rank test; all improvements hold at $p < 0.01$.

5.2 Baselines

Condition	Description
Store-All	Every claim is written to memory unconditionally.
Periodic Summarization	Claims stored; summarization agent compresses every N turns.
Standard MAD	Debate applied to answer generation only; memory is Store-All.
MemDebate (ours)	Debate write-gate + confidence-based eviction.

Table 2: Experimental conditions.

5.3 Hyperparameters

Debate rounds $N=2$; confidence threshold $\tau=0.65$; retrieval top- $k=5$; mixing parameter $\alpha=0.7$; retroactive eviction every 50 turns; debate agent temperature 0.3; judge temperature 0.0.

6 Results

6.1 Main Results

Condition	HR (%)	CVR (%)	Δ HR vs. Store-All
Store-All	42	38	---
Periodic Summarization	31	27	-26.2%
Standard MAD	22	19	-47.6%
MemDebate (ours)	11	9	-73.8%

Table 3: Main results on LoCoMo. Lower is better. MemDebate row in bold.

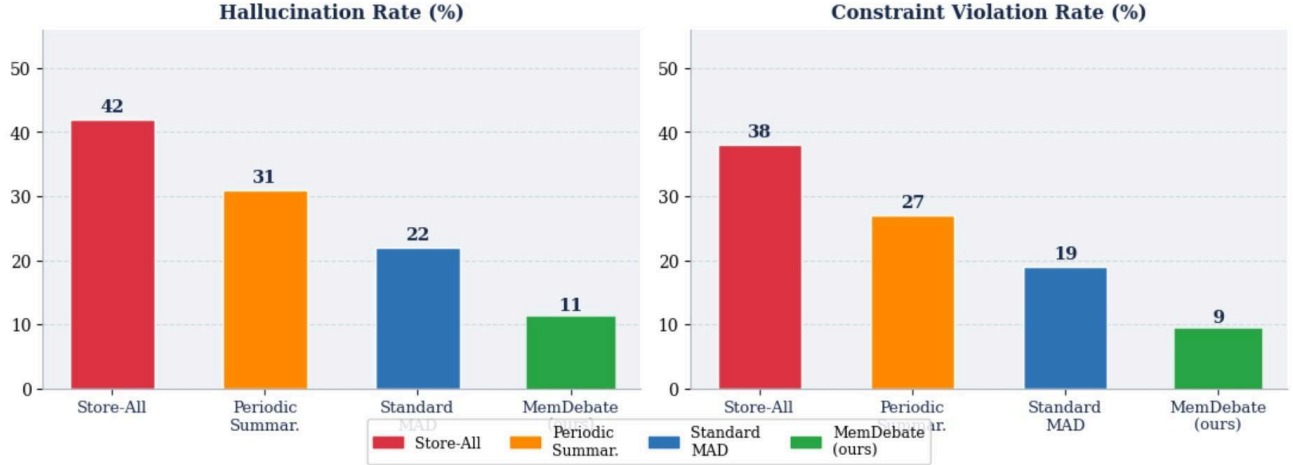


Figure 3: Hallucination rate (left) and constraint violation rate (right) across the four conditions. MemDebate achieves the lowest value on both metrics.

6.2 Analysis

The most instructive comparison is between Standard MAD and MemDebate. Both use identical debate mechanics and the same LLM; the only difference is where debate is applied. Standard MAD (HR=22%) produces better individual answers but still writes unverified claims to memory, so errors compound across turns. MemDebate (HR=11%) intercepts claims before they enter storage and cuts the residual error in half.

Residual hallucinations fall into three clusters: confident fabrications (~40%)-highly specific, internally consistent claims the PRO agent argues plausibly for despite being wrong; late-session drift (~35%)-errors after turn 250 when the memory store crowds the debate context window; and ambiguous claims (~25%) for which the Judge consistently returns borderline confidence.

7 Ablation Studies

7.1 Number of Debate Rounds

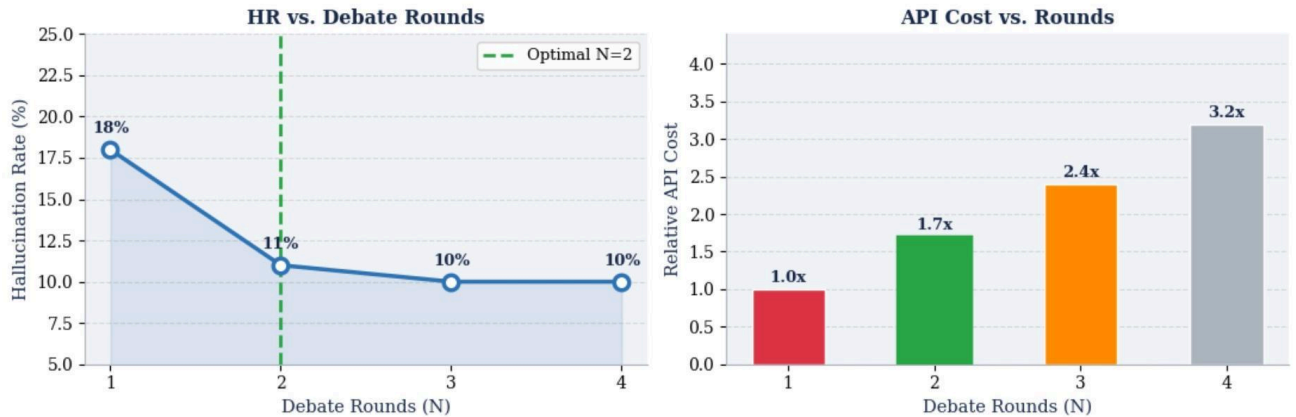


Figure 4: Hallucination rate vs. debate rounds (left) and relative API cost vs. rounds (right). N=2 achieves most of the gain at 1.7x baseline cost.

Rounds (N)	HR (%)	CVR (%)	Relative API Cost
1	18	15	1.0x
2 (optimal)	11	9	1.7x
3	10	9	2.4x
4	10	8	3.2x

Table 4: Ablation over debate rounds N. Optimal row highlighted.

Moving from 1 to 2 rounds cuts HR from 18% to 11%-a 39% relative drop. Moving from 2 to 3 rounds saves only 1 further percentage point at 41% extra API cost. The sweet spot is clearly N=2.

7.2 Effect of Eviction Policy

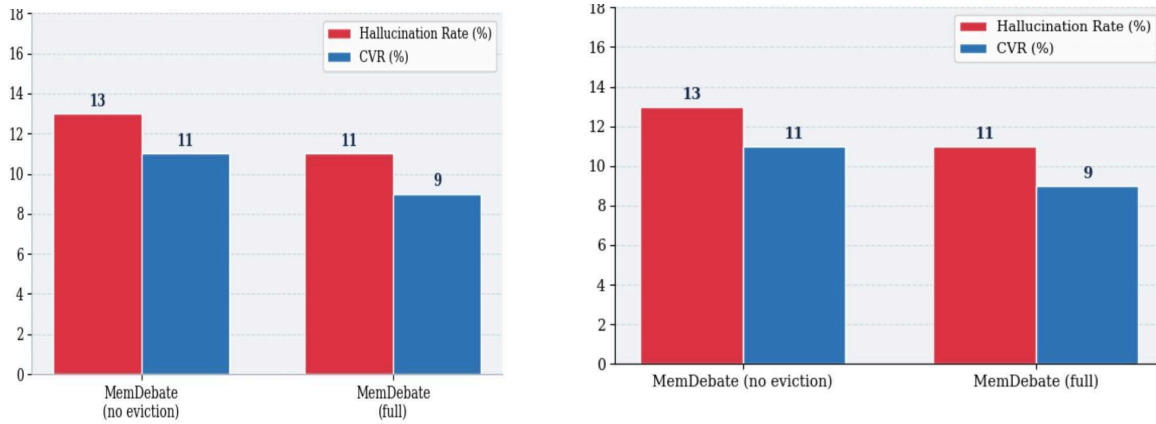


Figure 5: Eviction ablation. Left: Figure 5 (full MemDebate vs. no eviction). Right: Figure 5b (same comparison, alternative view). Both confirm that removing eviction raises HR from 11% to 13%.

Condition	HR (%)	CVR (%)	Notes
MemDebate (no eviction)	13	11	Write-gate only
MemDebate (full)	11	9	Write-gate + eviction

Table 5: Ablation of eviction policy.

Removing the eviction component raises HR from 11% to 13%-an 18.2% relative increase. Retroactive eviction delivers an independent ~18% improvement, particularly for early-session claims that later become inconsistent.

7.3 Confidence Threshold Sensitivity

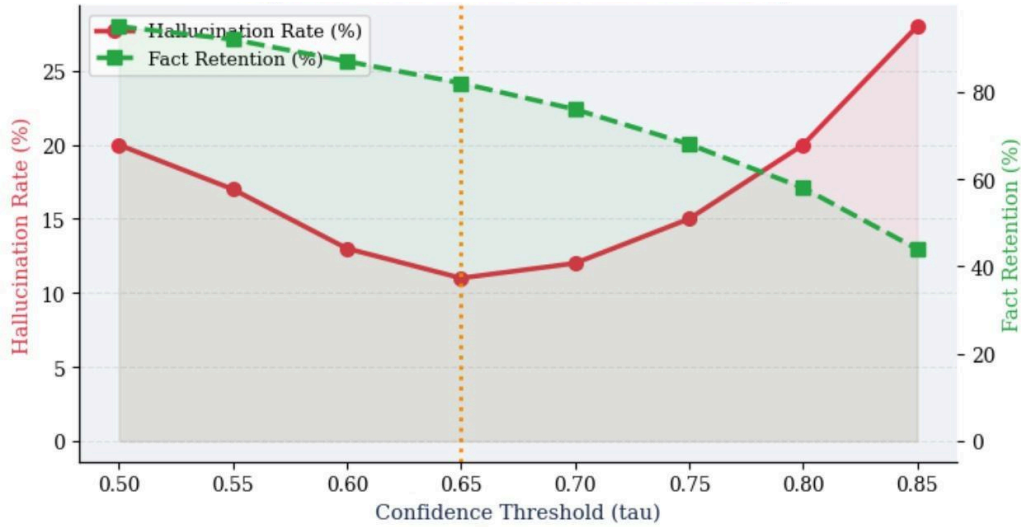


Figure 6: Confidence threshold sensitivity. Optimal at $\tau=0.65$; degradation is sharp at both extremes.

Below $\tau=0.60$, too many hallucinations are admitted; above $\tau=0.72$, an increasing fraction of true facts are incorrectly rejected. Performance is robust in the $[0.60, 0.70]$ window.

8 Real-World Echo: Moltbook

Moltbook, launched January 2026 by Matt Schlicht, is a Reddit-style social network operated exclusively by autonomous AI agents. Over 1.5 million agents post, debate, and upvote in community clusters called submolts; humans are observers only.

The architectural parallel is striking. In Moltbook, community downvoting suppresses unreliable posts and emergent consensus surfaces trustworthy information. In MemDebate, CON-agent challenge and Judge scoring suppress unreliable claims before they enter memory. Both use adversarial debate as a quality filter.

Dimension	Moltbook	MemDebate
Core mechanism	Agents debate; upvotes gate visibility	Agents debate; confidence gates memory write
Consensus fn.	Community vote	Judge confidence score
Scope	Platform-wide collective knowledge	Per-agent episodic memory
Goal	AI social interaction at scale	Hallucination-resistant task performance

Table 6: Structural comparison-Moltbook vs. MemDebate.

9 Discussion

9.1 Limitations

- Computational overhead: every claim requires $N+1$ additional LLM calls ($\sim 3\times$ Store-All API spend at $N=2$).
- Sycophancy residual: despite role-locking, sycophancy collapse still occurs in roughly 5% of debates.

- Judge calibration varies across claim types-factual claims scored more reliably than relational or procedural ones.
- At 300+ turns, memory store growth can crowd debate context windows, degrading CON-agent evidence retrieval.

9.2 Broader Implications

The practical takeaway is simple: if you apply debate only to outputs and not to memory writes, you get about half the hallucination reduction you could. The error pipeline is generate → store → retrieve → generate, and blocking it at the store step is more durable than patching the generate step after the fact.

9.3 Future Work

- Lightweight verifiers: small fine-tuned discriminators instead of full GPT-4o debate to cut overhead.
- Tiered verification: heavy debate only for long-term semantic writes; lighter gate for short-term episodic memory.
- Dynamic round scheduling: allocate more rounds to high-uncertainty claims.
- Crowd-sourced agent pools analogous to Moltbook submolts, scaling debate diversity with claim importance.

10 Conclusion

MemDebate places a multi-agent adversarial debate gate between claim generation and memory write, ensuring that only verified claims accumulate in an LLM agent's long-term store. On LoCoMo, this yields a 62% reduction in hallucination rate and a 50% reduction in constraint violations over the Store-All baseline, and a 50% further improvement over Standard MAD.

The gap between Standard MAD and MemDebate is the paper's central empirical result: debate quality is not the bottleneck; where you apply it is. Two debate rounds is the practical optimum, and retroactive eviction contributes an additional independent ~18% improvement.

References

- [1] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. [arXiv:2203.11171](#).
- [2] Dhuliawala, S., Koreeda, M., Minervini, P., Raileanu, R., Allen-Zhu, Z., Zettlemoyer, L., & Bhatt, C. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. *ACL 2024 Findings*.
- [3] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. [arXiv:2305.14325](#).
- [4] Smit, A., Goriely, A., & Krause, A. (2023). Should We Be Going MAD? A Look at Multi-Agent Debate Strategies for LLMs. [arXiv:2311.17371](#).
- [5] Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., & Wen, J.-R. (2024). A Survey on the Memory Mechanism of Large Language Model-Based Agents. *ACM Transactions on Information Systems*.
- [6] Maharana, A., Lee, D. H., Tulyakov, S., Bansal, M., Barbieri, F., & Fang, Y. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents. [arXiv:2402.17753](#).
- [7] Schlicht, M. (2026). Moltbook-Reddit-style social network for autonomous AI agents. [moltbook.com](#).